

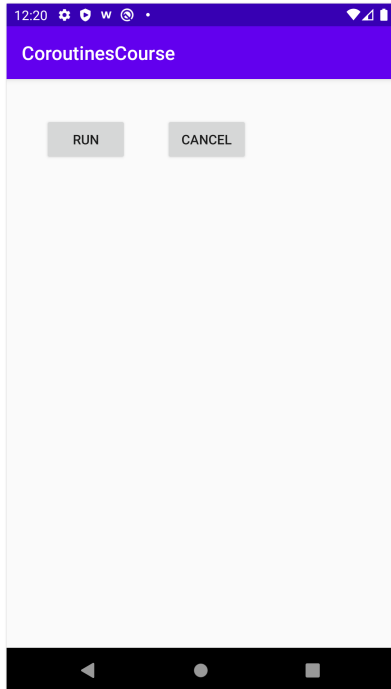
Урок 8. Корутины. Отмена

04 февраля 2020

В этом уроке запускаем и отменяем корутины. Проверяем, как корутины влияют на основной поток.

Много было теории в прошлых уроках, давайте пока на практику переключимся. Позапускаем корутины, посмотрим, как они себя ведут, и попробуем отменить их.

Для этого создайте новый проект и на экран добавьте пару кнопок: Run и Cancel



Добавьте dependencies для корутин

```
1 | implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-core:1.3.0'
2 | implementation 'org.jetbrains.kotlinx:kotlinx-coroutines-android:1.3.0'
```

Код Activity:

```

1 class MainActivity : AppCompatActivity() {
2
3     private var formatter = SimpleDateFormat("HH:mm:ss.SSS", Locale.getDefault())
4
5     private val scope = CoroutineScope(Job())
6
7     override fun onCreate(savedInstanceState: Bundle?) {
8         super.onCreate(savedInstanceState)
9         setContentView(R.layout.activity_main)
10
11         findViewById<View>(R.id.btnRun).setOnClickListener{
12             onRun()
13         }
14         findViewById<View>(R.id.btnCancel).setOnClickListener{
15             onCancel()
16         }
17     }
18
19     private fun onRun() {
20
21     }
22
23     private fun onCancel() {
24
25     }
26
27     override fun onDestroy() {
28         super.onDestroy()
29         log("onDestroy")
30         scope.cancel()
31     }
32
33     private fun log(text: String) {
34         Log.d("TAG", "${formatter.format(Date())} $text [${Thread.currentThread().name}]")
35     }
36 }
37

```

Мы уже знаем, что для запуска корутин нам нужен scope. И что его надо отменять, чтобы отменить все корутины в нем запущенные. Создаем самый простой вариант scope и в onDestroy добавляем его отмену.

По нажатию на кнопки вызываем методы onRun и onCancel. В этих методах и будем экспериментировать.

Логи будем выводить с указанием времени и потока.

Давайте запустим простую корутину в onRun. Она будет выполнять sleep и писать лог. Посмотрим, как она будет выполняться относительно основного кода.

```

1 private fun onRun() {
2     log("onRun, start")
3
4     scope.launch {
5         log("coroutine, start")
6         TimeUnit.MILLISECONDS.sleep(1000)
7         log("coroutine, end")
8     }
9
10    log("onRun, end")
11 }
12
13 private fun onCancel() {
14
15 }

```

В onCancel пока ничего не делаем.

Жмем кнопку Run, смотрим логи:

```

11:00:55.994 onRun, start [main]
11:00:56.062 onRun, end [main]
11:00:56.075 coroutine, start [DefaultDispatcher-worker-1]
11:00:57.076 coroutine, end [DefaultDispatcher-worker-1]

```

Логи показывают, что метод onRun полностью выполнен еще до того, как начала работать корутина. Т.е. билдер launch не блокирует поток, где он был запущен. Он создает корутину и отправляет ее на запуск в отдельном потоке (это видно по логам coroutine), а основной код идет дальше, и метод onRun завершается.

Корутина же продолжает выполнение. О том, в каком именно потоке это происходит, и как задать свой поток, мы поговорим отдельно в уроке про диспетчеры. Сейчас нам пока достаточно знать, что используется фоновый поток из диспетчера по умолчанию.

Добавим еще одну корутину, чтобы посмотреть, будут ли две корутины как-то ждать или блокировать друг друга

```

1 private fun onRun() {
2     log("onRun, start")
3
4     scope.launch {
5         log("coroutine, start")
6         TimeUnit.MILLISECONDS.sleep(1000)
7         log("coroutine, end")
8     }
9
10    log("onRun, middle")
11
12    scope.launch {
13        log("coroutine2, start")
14        TimeUnit.MILLISECONDS.sleep(1500)
15        log("coroutine2, end")
16    }
17
18    log("onRun, end")
19 }

```

Смотрим логи:

```

11:07:54.350 onRun, start [main]
11:07:54.351 onRun, middle [main]
11:07:54.352 onRun, end [main]
11:07:54.352 coroutine, start [DefaultDispatcher-worker-4]
11:07:54.354 coroutine, start [DefaultDispatcher-worker-1]
11:07:55.355 coroutine, end [DefaultDispatcher-worker-1]
11:07:55.855 coroutine2, end [DefaultDispatcher-worker-4]

```

Метод onRun быстро выполнялся, создав и отправив на выполнение пару корутин. Корутины стартовали практически одновременно, в отдельных потоках сделали свою работу и завершились. Между собой они никак не связаны.

Эти примеры показывают, что билдер launch быстро выполняется, не блокируя и не задерживая основной поток. А корутины уходят в отдельный поток и выполняются там. Это самый простой случай запуска корутин. Он не особо отличается от ручного создания потока и передачи туда Runnable на выполнение. Позже мы будем связывать корутины между собой и станет понятно, в чем их преимущество перед другими асинхронными механизмами.

Отмена корутины через job

Давайте попробуем отменить корутину с помощью ее же джоба. Для этого у Job есть метод cancel.

```

1 lateinit var job: Job
2
3 private fun onRun() {
4     log("onRun, start")
5
6     job = scope.launch {
7         log("coroutine, start")
8         var x = 0
9         while (x < 5) {
10             TimeUnit.MILLISECONDS.sleep(1000)
11             log("coroutine, ${x++}")
12         }
13         log("coroutine, end")
14     }
15
16     log("onRun, end")
17 }
18
19 private fun onCancel() {
20     log("onCancel")
21     job.cancel()
22 }

```

В onRun запускаем корутину и получаем ее джоб. В корутине несколько раз запускаем паузу в цикле. По нажатию на Cancel отменяем корутину.

Жмем Run, а через пару секунд - Cancel.

```

11:24:53.416 onRun, start [main]
11:24:53.466 onRun, end [main]
11:24:53.474 coroutine, start [DefaultDispatcher-worker-1]
11:24:54.475 coroutine, 0 [DefaultDispatcher-worker-1]
11:24:55.476 coroutine, 1 [DefaultDispatcher-worker-1]
11:24:56.135 onCancel [main]
11:24:56.480 coroutine, 2 [DefaultDispatcher-worker-1]
11:24:57.484 coroutine, 3 [DefaultDispatcher-worker-1]
11:24:58.486 coroutine, 4 [DefaultDispatcher-worker-1]
11:24:58.486 coroutine, end [DefaultDispatcher-worker-1]

```

Корутина продолжила выполняться несмотря на отмену. Так произошло потому, что метод cancel меняет статус корутины, но не прерывает ее выполнение. А от нас требуется отслеживать этот статус и самим завершать работу корутины. Для этого используется поле isActive.

Добавим это поле в логи и в проверку цикла.

```

1 | lateinit var job: Job
2 |
3 | private fun onRun() {
4 |     log("onRun, start")
5 |
6 |     job = scope.launch {
7 |         log("coroutine, start")
8 |         var x = 0
9 |         while (x < 5 && isActive) {
10 |             TimeUnit.MILLISECONDS.sleep(1000)
11 |             log("coroutine, ${x++}, isActive = ${isActive}")
12 |         }
13 |         log("coroutine, end")
14 |     }
15 |
16 |     log("onRun, end")
17 | }
18 |
19 | private fun onCancel() {
20 |     log("onCancel")
21 |     job.cancel()
22 | }

```

Если isActive начнет возвращать false, то мы прерываем цикл.

Смотрим логи:

```

11:27:45.001 onRun, start [main]
11:27:45.048 onRun, end [main]
11:27:45.053 coroutine, start [DefaultDispatcher-worker-1]
11:27:46.053 coroutine, 0, isActive = true [DefaultDispatcher-worker-1]
11:27:47.055 coroutine, 1, isActive = true [DefaultDispatcher-worker-1]
11:27:47.665 onCancel [main]
11:27:48.057 coroutine, 2, isActive = false [DefaultDispatcher-worker-1]
11:27:48.057 coroutine, end [DefaultDispatcher-worker-1]

```

После нажатия на кнопку Cancel, метод isActive начинает возвращать false. Это означает, что корутина больше не активна. Ее выполнение или ее результат более никому не нужны. Можно прервать ее работу, что мы и делаем в условии цикла.

Если мы посмотрим на исходники isActive, то увидим, что из контекста достается Job корутины и читается его поле isActive.

```

1 | public val CoroutineScope.isActive: Boolean
2 |     get() = coroutineContext[Job]?.isActive ?: true

```

Т.е. методом cancel мы снаружи корутины меняем статус ее джоба, а поле isActive сообщает нам об этом уже внутри корутины.

Отмена корутины через scope

В прошлом уроке мы обсуждали, что отменить корутины можно, используя scope, в котором они были созданы. Напомню, что при создании корутины ее джоб подписывается на джоб из scope. Когда мы отменяем scope, мы, тем самым, отменяем его джоб. А он уже отменяет все подписанные на него джобы, т.е. корутины.

Давайте проверим это. В onDestroy у нас висит отмена скопа. Попробуем нажать кнопку Run и через пару секунд закрыть приложение.

```

11:32:14.242 onRun, start [main]
11:32:14.284 onRun, end [main]
11:32:14.293 coroutine, start [DefaultDispatcher-worker-1]
11:32:15.294 coroutine, 0, isActive = true [DefaultDispatcher-worker-1]
11:32:16.295 coroutine, 1, isActive = true [DefaultDispatcher-worker-1]
11:32:16.355 onDestroy [main]
11:32:17.297 coroutine, 2, isActive = false [DefaultDispatcher-worker-1]
11:32:17.297 coroutine, end [DefaultDispatcher-worker-1]

```

Логи показывают, что корутина была отменена. Scope сделал свое дело.

Для примера добавьте запуск еще нескольких корутин, используя этот scope, и они все будут отменены при закрытии приложения.

suspend функции

Когда внутри корутины используется suspend функция, поведение корутины при отмене может отличаться от того, что мы рассмотрели выше. Попробуйте заменить TimeUnit.MILLISECONDS.sleep(1000) на delay(1000). Эта suspend функция также делает паузу в указанное количество миллисекунд.

При запуске и отмене корутины логи будут следующими:

```

20:29:33.578 onRun, start [main]
20:29:33.606 onRun, end [main]
20:29:33.612 coroutine, start [DefaultDispatcher-worker-1]
20:29:34.619 coroutine, 0, isActive = true [DefaultDispatcher-worker-1]
20:29:35.622 coroutine, 1, isActive = true [DefaultDispatcher-worker-3]
20:29:36.201 onCancel [main]

```

Отличие от предыдущих примеров в том, что функция `delay` при отмене корутины сразу прерывает выполнение кода корутины. Метод `log` после `delay` не вызывается и до проверки `isActive` в цикле уже не доходит.

Мы еще поговорим, как это работает, когда будем изучать обработку ошибок в корутинах. Если вкратце, то функция `delay` подписывается на событие отмены корутины, и при возникновении такого события бросает специальный `Exception`. Именно поэтому выполнение кода корутины прерывается на `delay` и дальше не идет. Но этот `Exception` ловится и обрабатывается внутренними механизмами корутины, поэтому никакого крэша не происходит.

Такую специфику имеет не все `suspend` функции, а только `cancellable`. Они создаются аналогично тому, как мы проходили в уроке про `suspend` функции, но вместо `suspendCoroutine` надо использовать `suspendCancellableCoroutine`.